



MyCodeSchool Matrici — fisa 2

Chenare · Stergeri · Inserari · Interschimbări

Enuntul general: se citeste o matrice de maxim 100 x 100 elemente, numere intregi. Fiecare cerinta se rezolva intr-o functie separata, apelata din `main`, si se testeaza pe rand.

Ce ai voie sa folosesti: tot ce ai deja in `functii.h` si `solutii.h` — `citireMatrice`, `afisareMatrice`, `stergereLinie`, `stergereColoana`, `inserareLinie`, `inserareColoana`, `interschimbareLinii`, `afisareChenar`, `isNumarPrim`, `isNumarPp`, `oglindit`, `cifraControl`, `aparitiiCifraLinie`, `linieCrescatoare`, `linieDescrescatoare`.

Regula fisei: daca ai nevoie de o bucla pe care ai scris-o deja in alta functie, **apeleaza functia**, nu copia bucla. Daca iti trebuie o varianta pe coloane a unei functii pe linii, scrie-o alaturi, cu acelasi tipar.

Inainte de fiecare commit: Build (F9). Daca nu compileaza, nu se comite.

Inca NU folosim: `vector`, `string`, `<algorithm>`, `struct`, `recursivitate`.

/** Chenarele unei matrici */

a) Suma fiecarui chenar

usor

Afisati suma elementelor de pe fiecare chenar exterior, incepand cu gradul 0 si pana la ultimul chenar care exista in matrice, cate o suma pe linie.

```
matricea:
 1  2  3  4  5
 6  7  8  9 10
11 12 13 14 15
16 17 18 19 20
iese:
Chenar 0: 147
Chenar 1: 63
```

Cand se opreste bucla peste grade? Scrie conditia pornind de la `i1`, `j1`, `i2`, `j2`, nu ghici un numar.

b) Numere prime pe chenarul de grad k

usor

Se citeste `k` de la tastatura. Afisati cate numere prime se afla pe chenarul exterior de grad `k`.
Daca matricea nu are un chenar de grad `k`, se afiseaza mesajul `NU EXISTA`.

```
matricea de la a), k = 1
iese: 2
pentru k = 2
iese: NU EXISTA
```

Chenarul de grad 1 al matricei de mai sus e 7 8 9 12 13 14. Testeaza si pe o matrice 3x3 cu `k=1` — acolo chenarul are un singur element.

c) Chenarul cu suma maxima

mai greu

Determinati gradul chenarului cu suma maxima a elementelor si inlocuiti toate elementele acelui chenar cu cifra de control a sumei lui.

```
matricea de la a) – chenar 0 are suma 147, chenar 1 are 63  
cifra de control a lui 147 este 3, deci iese:
```

```
3 3 3 3 3  
3 7 8 9 3  
3 12 13 14 3  
3 3 3 3 3
```

/*** Stergeri ***/

d) Coloane cu palindromuri de 3 cifre

usor

Stergeti toate coloanele care contin cel putin un element palindrom cu exact 3 cifre.

```
intra:  
121 45 78 9  
12 232 78 33  
15 45 100 7  
iese:  
78 9  
78 33  
100 7
```

33 e palindrom, dar are 2 cifre. 100 are 3 cifre, dar nu e palindrom.

e) Ultimul patrat perfect

usor

Stergeti linia si coloana pe care se afla ultimul patrat perfect din matrice (parcurete pe linii, de sus in jos si de la stanga la dreapta). Daca matricea nu contine niciun patrat perfect, se afiseaza NU EXISTA .

```
intra:  
5 16 7 20  
9 11 30 12  
3 14 25 8  
ultimul patrat perfect este 25, pe linia 2, coloana 2 – iese:  
5 16 20  
9 11 12
```

f) Linii cu cifra c cel putin cat prima linie

mai greu

Se citeste cifra `c`. Stergeti toate liniile in care cifra `c` apare de cel putin la fel de multe ori ca in prima linie. Daca in final matricea ramane fara linii, se afiseaza `MATRICE VIDA`.

```
intra, c = 1
11 20 35      cifra 1 apare de 2 ori
41 15 71      de 3 ori
22 30 44      de 0 ori
10 111 25     de 4 ori
prima linie are 2 aparitii, deci iese:
22 30 44
```

Prima linie se sterge si ea. Cand calculezi numarul de referinta si cand il folosesti? Ce se intampla daca il recalculezi dupa fiecare stergere?

/**/ Inserari /**/

g) Dupa fiecare linie numai cu elemente pare

usor

Inserati dupa fiecare linie care are toate elementele pare, linia formata din cifrele de control ale elementelor ei.

```
intra:
12 48 60
15 22 31
84 26 70
iese:
12 48 60
3 3 6
15 22 31
84 26 70
3 8 7
```

Dupa ce ai inserat o linie noua, pe ce pozitie se afla urmatoarea linie de verificat? Ai mai avut situatia asta la `solutieM()`.

h) Inaintea coloanei cu suma minima

usor

Inserati inainte de coloana cu cea mai mica suma a elementelor, coloana formata din oglinditul elementelor ei. Daca mai multe coloane au aceeasi suma minima, se ia prima dintre ele.

```
intra:          sumele: 67  77  139
 12  45  70
 31  22  13
 24  10  56
iese:
21  12  45  70
13  31  22  13
42  24  10  56
```

i) Dupa ultima linie descrescatoare

mai greu

Inserati dupa ultima linie care are elementele in ordine descrescatoare, o linie in care fiecare element este numarul de divizori proprii ai elementului de deasupra lui. Daca nu exista o astfel de linie, se afiseaza **NU**.

```
intra:
 30  20  10
  5  12  40
 60  18   6      ultima linie descrescatoare
 12  12  15
iese:
 30  20  10
  5  12  40
 60  18   6
 10   4   2
 12  12  15
```

Divizori proprii = divizorii diferiti de 1 si de numarul insusi. 60 are 10, 18 are 4, 6 are 2.

/** Interschimbare */

j) Coloana crescatoare cu coloana descrescatoare

usor

Interschimbati prima coloana care are elementele in ordine crescatoare cu ultima coloana care are elementele in ordine descrescatoare. Daca una dintre ele nu exista, se afiseaza **NU SE POATE**.

```
intra:                coloana 0 e crescatoare, coloana 3 e descrescatoare
1  9  5  8
3  6  2  6
4  4  9  5
7  2  7  1
iese:
8  9  5  1
6  6  2  3
5  4  9  4
1  2  7  7
```

Ai nevoie de `interschimbareColoane` si de o varianta pe coloane a lui `linieCrescatoare`. Scrie-le cu acelasi tipar, in `functii.h` — nu copia bucele in interiorul solutiei.

k) Linii doua cate doua

usor

Interschimbati intre ele liniile doua cate doua: linia 0 cu linia 1, linia 2 cu linia 3 si asa mai departe. Daca numarul de linii este impar, ultima linie ramane pe loc.

```
intra:                iese:
1  2  3                4  5  6
4  5  6                1  2  3
7  8  9                10 11 12
10 11 12               7  8  9
13 14 15               13 14 15
```

I) Linie cu coloana

provocare

Interschimbati linia cu cele mai multe elemente prime cu coloana cu cele mai multe elemente prime, element cu element: `a[linie][j]` cu `a[j][coloana]`, in ordinea `j = 0, 1, 2, ...`.
Daca matricea nu este patratica, se afiseaza **NU SE POATE**. Daca mai multe linii (sau coloane) au acelasi numar de prime, se ia prima dintre ele.

```
intra:          linia 2 are 4 prime, coloana 0 are 2 prime
2  8  3  9
4  6 10 12
5  7 11 13
8 14  6 15
iese:
5  8  3  9
7  6 10 12
11 4  2  8
13 14 6 15
```

Ce se intampla cu elementul aflat la intersectia liniei cu coloana? Urmareste-l pas cu pas, pentru fiecare `j`, si vei intelege de ce rezultatul nu e o simpla oglindire.